TCS NQT Coding Problem – Full Solution Format

1. Problem Title & Source

Title: **Count Number of Sundays**

Source: **TCS NQT Previous Coding Questions**

2. Problem Statement (Short Summary)

Jack loves Sundays because it is his favorite day of the week.

You are given:

- The **starting day of a month**
- The **number of days n**

The month may start on any day:

sun, mon, tue, wed, thu, fri, sat

Your task is to **count how many Sundays occur within the first n days** of that month.

Key idea:

Every 7 days → Sunday repeats

3. Examples

Example 1

Input

mon

13

Output

1

Explanation

If the month starts on **Monday**, the days will be:

1 Mon

2 Tue

3 Wed

4 Thu

5 Fri

6 Sat

7 Sun ← first Sunday

8 Mon

9 Tue

10 Wed

11 Thu

12 Fri

13 Sat

Only **day 7** is Sunday.

Therefore:

Total Sundays = 1

Example 2

Input

sun

10

Output

2

Explanation

1 Sun

2 Mon

3 Tue

4 Wed

5 Thu

6 Fri

7 Sat

8 Sun

9 Mon

10 Tue

Sundays:

Day 1

Day 8

Total Sundays = **2**

4. Constraints

$1 \le n \le 31$

Valid days:

sun, mon, tue, wed, thu, fri, sat

5. Core Concept (Pattern)

Math + Simulation

Important observation:

Sunday occurs every 7 days

6. Thought Process

Step 1 — Convert Day to Index

Assign numeric values:

sun → 0

mon → 1

tue → 2

wed → 3

thu → 4

fri → 5

sat → 6

Step 2 — Find First Sunday

If month starts on day d:

firstSunday = (7 - d) % 7

But since **days start from 1**, the actual Sunday position is:

firstSundayDay = firstSunday + 1

Step 3 — Count Sundays

After the first Sunday:

every 7 days

7. Visualization

Example

start = mon

n = 13

Day mapping:

mon = 1

First Sunday:

(7 - 1) % 7 = 6

Actual day:

6 + 1 = 7

Next Sundays:

7, 14, 21 ...

Within 13 days:

7

Count = **1**

8. Pseudocode

```
read start_day
read n

map days to numbers

d = mapped value

firstSunday = (7 - d) % 7
day = firstSunday + 1

count = 0

while day <= n
   count++
   day += 7

print count
```

9. Code Implementation

Python

```
day = input().strip().lower()
n = int(input())

days = {
   "sun":0,"mon":1,"tue":2,"wed":3,
   "thu":4,"fri":5,"sat":6
}

d = days[day]

first_sunday = (7 - d) % 7
day = first_sunday + 1

count = 0

while day <= n:
```

```
    count += 1
    day += 7

print(count)
```

Java

```
import java.util.*;

class Main {

    public static void main(String[] args){

        Scanner sc = new Scanner(System.in);

        String day = sc.next().toLowerCase();
        int n = sc.nextInt();

        Map<String,Integer> map = new HashMap<>();

        map.put("sun",0);
        map.put("mon",1);
        map.put("tue",2);
        map.put("wed",3);
        map.put("thu",4);
        map.put("fri",5);
        map.put("sat",6);

        int d = map.get(day);

        int firstSunday = (7 - d) % 7;
        int sunday = firstSunday + 1;

        int count = 0;

        while(sunday <= n){
            count++;
            sunday += 7;
```

```
        }

        System.out.println(count);
    }
}
```

10. Time & Space Complexity

Time Complexity

O(n/7)

Maximum about **5 iterations**.

Space Complexity

O(1)

11. Common Mistakes

1. Off-by-one errors
2. Wrong day mapping
3. Counting day 0
4. Forgetting modulo operation

12. Real-Life Applications

Calendar based calculations:

- Event scheduling
- Meeting recurrence
- Weekly analytics
- Date simulations